

INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Pós-Graduação em Sistemas Computacionais Embebidos
Sistemas Operativos de Tempo-Real (PGSCE-SIOPTR)

RTOSP: TT2: Week 3

Linux Kernel Modules

Paulo Baltarejo Sousa
pbs@isep.ipp.pt
2025/26

1 Learning examples

Download the learning examples from <https://drive.google.com/file/d/1P4ZtPu4W5kQZ58P8FgcU-cXwsAkfMqZ/view?usp=sharing>. Download LKM1.tar.gz compressed file. In this compressed file you can find several source code modules and a Makefile.

For extracting it type:

```
> tar -xvzf LKM1.tar.gz
```

1.1 hello.c

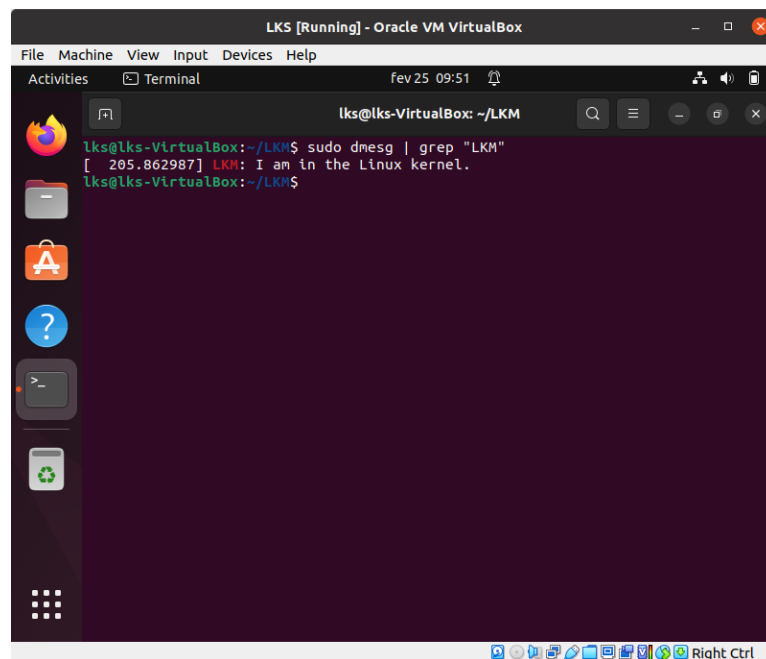
This is the simplest Linux kernel module. Check the code and then:

1. For compiling it, type:

```
> make FILE=hello
```
2. Insert it into the Linux kernel:

```
> sudo insmod ./hello.ko
```
3. Check printk messages:

```
> sudo dmesg | grep "LKM"
```



4. Check list modules:

```
> lsmod
```

The screenshot shows a terminal window titled "lks@lks-VirtualBox: ~/LKM". The user has run the command `lsmod`, which lists loaded kernel modules. The output is as follows:

Module	Size	Used by
hello	16384	0
vboxsf	81920	1
vboxvideo	45056	0
snd_intel8x0	49152	2
snd_ac97_codec	176128	1 snd_intel8x0
ac97_bus	16384	1 snd_ac97_codec
binfmt_misc	24576	1
snd_pcm	155648	2 snd_intel8x0,snd_ac97_codec
snd_seq_midi	20480	0
snd_seq_midi_event	16384	1 snd_seq_midi
snd_rawmidi	45056	1 snd_seq_midi
nls_iso8859_1	16384	1
intel_rapl_msr	20480	0
snd_seq	77824	2 snd_seq_midi,snd_seq_midi_event
intel_rapl_common	40960	1 intel_rapl_msr
intel_powerclamp	24576	0
crc10dif_pclmul	16384	1
ghash_clmulni_intel	16384	0
snd_seq_device	16384	3 snd_seq,snd_seq_midi,snd_rawmidi
snd_timer	40960	2 snd_seq,snd_pcm
aesni_intel	376832	0
crypto_simd	16384	1 aesni_intel
cryptd	24576	2 crypto_simd,ghash_clmulni_intel
rapl	20480	0
joydev	32768	0

5. Remove it from the Linux kernel: `> sudo rmmod hello`

6. Check printk messages: `> sudo dmesg | grep "LKM"`

The screenshot shows the same terminal window after the user has run `sudo rmmod hello` and `sudo dmesg | grep "LKM"`. The output is as follows:

```
lks@lks-VirtualBox: ~/LKM$ sudo rmmod hello
lks@lks-VirtualBox: ~/LKM$ sudo dmesg | grep "LKM"
[ 205.862987] LKM: I am in the Linux kernel.
[ 507.231849] LKM: I am no longer in the Linux kernel.
lks@lks-VirtualBox: ~/LKM$
```

1.2 hello2.c

This is a basic module that create a new entry in the `proc` directory. Further, it uses `proc_ops` data structure.

1. Compilation:
> `make FILE=hello2`
2. Insert it into the Linux kernel:
> `sudo insmod ./hello2.ko`
3. Change `/proc/hello2` permissions:
> `sudo chmod 666 /proc/hello2`
4. Perform read operation:
> `cat /proc/hello2`
> `sudo dmesg`
5. Perform write operation:
> `echo "aa" > /proc/hello2`
> `sudo dmesg`
6. Remove it from the Linux kernel:
> `sudo rmmod hello2`
7. Check printk messages:
> `sudo dmesg`

1.3 hello3.c

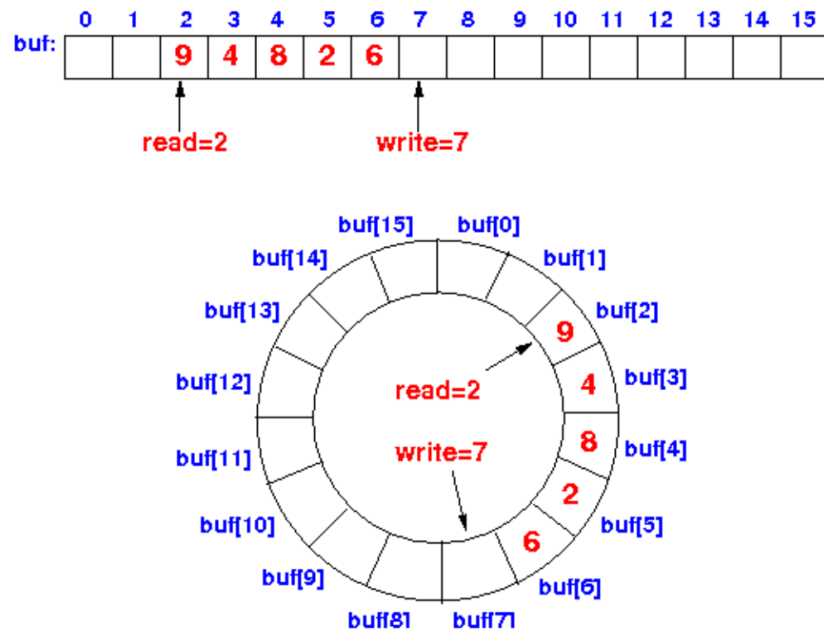
This module allows to read and write messages from/to kernel.

1. Compilation:
> `make FILE=hello3`
2. Insert it into the Linux kernel:
> `sudo insmod ./hello3.ko`
3. Change `/proc/hello3` permissions:
> `sudo chmod 666 /proc/hello3`
4. Perform read operation:
> `cat /proc/hello3`

5. Perform write operation:
`> echo "LKM is cool" > /proc/hello3`
6. Perform read operation:
`> cat /proc/hello3`
7. Remove it from the Linux kernel:
`> sudo rmmmod hello3`
8. Check printk messages:
`> sudo dmesg`

1.4 ring_buffer.c

Circular Queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".



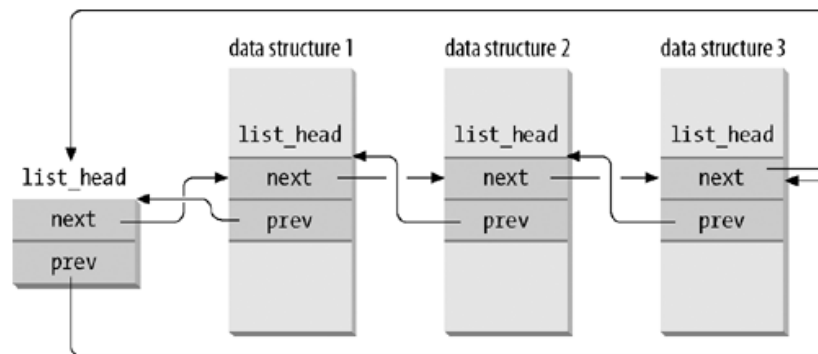
This module implements a ring buffer.

1. Compilation: `> make FILE=ring_buffer`
2. Insert it into the Linux kernel:
`> sudo insmod ./ring_buffer.ko`

3. Change `/proc/ring_buffer` permissions:
`> sudo chmod 666 /proc/ring_buffer`
4. Test it by performing a set of write and read operations:
`> echo "1" > /proc/ring_buffer`
`> echo "2" > /proc/ring_buffer`
`> echo "3" > /proc/ring_buffer`
`> echo "4" > /proc/ring_buffer`
`> ...`
`> cat /proc/ring_buffer`
`> cat /proc/ring_buffer`
`> cat /proc/ring_buffer`
`> cat /proc/ring_buffer`
5. Remove it from the Linux kernel:
`> sudo rmmod ring_buffer`
6. Check printk messages:
`> sudo dmesg`

1.5 `fifo_queue.c`

This module implements a FIFO queue through a linked list.



1. Compilation: `> make FILE=fifo_queue`
2. Insert it into the Linux kernel:
`> sudo insmod ./fifo_queue.ko`
3. Change `/proc/fifo_queue` permissions:
`> sudo chmod 666 /proc/fifo_queue`

4. Test it by performing a set of write and read operations:

```
> echo "1" > /proc/fifo_queue  
> echo "2" > /proc/fifo_queue  
> echo "3" > /proc/fifo_queue  
> echo "4" > /proc/fifo_queue  
> ...  
> cat /proc/fifo_queue  
> cat /proc/fifo_queue  
> cat /proc/fifo_queue  
> cat /proc/fifo_queue
```

5. Remove it from the Linux kernel:

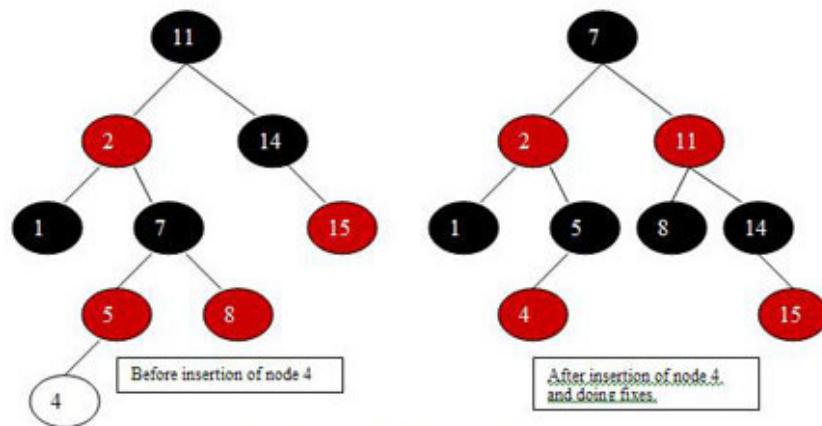
```
> sudo rmmmod fifo_queue
```

6. Check printk messages:

```
> dmesg
```

1.6 sorted_queue.c

A red-black tree is a kind of self-balancing binary search tree in computer science. Each node of the binary tree has an extra bit, and that bit is often interpreted as the color (red or black) of the node. These color bits are used to ensure the tree remains approximately balanced during insertions and deletions. This module implements a sorted queue through a red-black tree list.



1. Compilation: `> make FILE=sorted_queue`

2. Insert it into the Linux kernel:


```
> sudo insmod ./sorted_queue.ko
```
3. Change /proc/sorted_queue permissions:


```
> sudo chmod 666 /proc/sorted_queue
```
4. Test it by performing a set of write and read operations:


```
> echo "5" > /proc/sorted_queue
> echo "8" > /proc/sorted_queue
> echo "3" > /proc/sorted_queue
> echo "7" > /proc/sorted_queue
> ...
> cat /proc/sorted_queue
> cat /proc/sorted_queue
> cat /proc/sorted_queue
> cat /proc/sorted_queue
```
5. Remove it from the Linux kernel:


```
> sudo rmmod sorted_queue
```
6. Check printk messages:

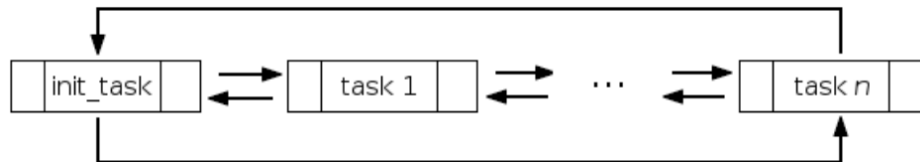

```
> sudo dmesg
```

1.7 tasklist.c

This module gets information about all tasks/processes that are active in the kernel. All tasks/processes are managed by the kernel using a data structure called `struct task_struct`. This data structure contains fields for everything that is needed to manage each task internally. The `struct task_struct` is a relatively large data structure, at around 1.7 kilobytes on a 32-bit machine. This size, however, is quite small considering that the structure contains all the information that the kernel has and needs about a process.

```
struct task_struct {
    ...
    struct list_head tasks ;
    ...
    pid_t pid ;
    ...
    struct task_struct __rcu * parent ; // recipient of SIGCHLD , wait4 () reports
    // children / sibling forms the list of my natural children
    struct list_head children ; // list of my children
    struct list_head sibling ; // linkage in my parent 's children list
    char comm [ TASK_COMM_LEN ] ; // executable name excluding path ...
    .
}
```

The kernel stores the list of tasks/processes in a circular doubly linked list called the task list. `init_task` (defined in `/init/init_task.c`) is the beginning of the list, although strictly speaking there is no head since this is a circular list.



1. Compilation: `> make FILE=tasklist`
2. Insert it into the Linux kernel:
`> sudo insmod ./tasklist.ko`
3. Change `/proc/tasklist` permissions:
`> sudo chmod 666 /proc/tasklist`
4. Enter:
`> cat /proc/tasklist`
5. Check printk messages:
`> sudo dmesg`
6. Remove it from the Linux kernel:
`> sudo rmmod tasklist`

1.8 atomic.c

Atomic operations provide instructions that execute atomically without interruption. Just as the atom was originally thought to be an indivisible particle, atomic operators are indivisible instructions.

1. Compilation: `> make FILE=atomic`
2. Insert it into the Linux kernel:
`> sudo insmod ./atomic.ko`
3. Change `/proc/atomic` permissions:
`> sudo chmod 666 /proc/atomic`

4. Test it by performing a set of write and read operations (using more than one terminal):

```
> echo " " > /proc/atomic
> echo " " > /proc/atomic
> echo " " > /proc/atomic
> echo " " > /proc/atomic
> ...
> cat /proc/atomic
> cat /proc/atomic
> cat /proc/atomic
> cat /proc/atomic
```
5. Check printk messages:

```
> sudo dmesg
```
6. Remove it from the Linux kernel:

```
> sudo rmmod atomic
```

1.9 spin.c

Spinlocks are simple in concept. A spinlock is a mutual exclusion device that can have only two values: **locked** and **unlocked**. It is usually implemented as a single bit in an integer value. Code wishing to take out a particular lock tests the relevant bit. If the lock is available, the **locked** bit is set and the code continues into the critical section. If, instead, the lock has been taken by somebody else, the code goes into a tight loop where it repeatedly checks the lock until it becomes available. This loop is the "spin" part of a spinlock.

1. Compilation:

```
> make FILE=spin
```
2. Insert it into the Linux kernel:

```
> sudo insmod ./spin.ko
```
3. Change /proc/spin permissions:

```
> sudo chmod 666 /proc/spin sud
```
4. Open several consoles and type:

```
> echo " " > /proc/spin
and
> cat /proc/spin
```

5. Check printk messages:
 > `sudo dmesg`
6. Remove it from the Linux kernel:
 > `sudo rmmod spin`

1.10 sema.c

This module shows how to use `struct semaphore` variables.

1. Compilation: > `make FILE=sema`
2. Insert it into the Linux kernel:
 > `sudo insmod ./sema.ko`
3. Change `/proc/sema` permissions:
 > `sudo chmod 666 /proc/sema`
4. Open several consoles and type:
 > `cat /proc/sema`
 > `echo " " > /proc/sema`
5. Check printk messages:
 > `sudo dmesg`
6. Remove it from the Linux kernel:
 > `sudo rmmod sema`

1.11 mut.c

This module shows how to use `struct mutex` variables.

1. Compilation: > `make FILE=mut`
2. Insert it into the Linux kernel:
 > `sudo insmod ./mut.ko`
3. Change `/proc/mut` permissions:
 > `sudo chmod 666 /proc/mut`
4. Open several consoles and type:
 > `cat /proc/mut`
 > `echo " " > /proc/mut`

5. Check printk messages:
 `> sudo dmesg`
6. Remove it from the Linux kernel:
 `> sudo rmmmod mut`

1.12 waitq.c

This module shows how to use waitqueues variables.

1. Compilation: `> make FILE=waitq`
2. Insert it into the Linux kernel:
 `> sudo insmod ./waitq.ko`
3. Change `/proc/waitq` permissions:
 `> sudo chmod 666 /proc/waitq`
4. Open several consoles and type:
 `> cat /proc/waitq`
 `> echo " " > /proc/waitq`
5. Check printk messages:
 `> sudo dmesg`
6. Remove it from the Linux kernel:
 `> sudo rmmmod waitq`

1.13 compl.c

This module shows how to use completion variables.

1. Compilation: `> make FILE=compl`
2. Insert it into the Linux kernel:
 `> sudo insmod ./compl.ko`
3. Change `/proc/compl` permissions:
 `> sudo chmod 666 /proc/compl`
4. Open several consoles and type:
 `> cat /proc/compl`
 `> echo " " > /proc/compl`

5. Check printk messages:
 > `sudo dmesg`
6. Remove it from the Linux kernel:
 > `sudo rmmod compl`